Due date: December 12, 2025
Late submission: 20% per day.
Teams: You can do the project in teams of 2. Teams must submit only 1 copy of the project.

Comparative Study of Simulated Annealing and CSP Backtracking for Sudoku Puzzle Solving

Project Overview:

Sudoku is a widely known logic-based puzzle that serves as an excellent benchmark for testing AI search algorithms. The objective is to fill a 9×9 grid so that every row, column, and 3×3 subgrid contains all digits from 1 to 9 exactly once. In this project, you are required to formulate Sudoku as both a Constraint Satisfaction Problem (CSP) and as an optimization problem, Implement and analyze CSP Backtracking Search and Simulated Annealing (SA) for solving Sudoku puzzles, and Compare their solution quality, speed, and robustness on puzzles of varying difficulty.

Problem Description:

Input:
- A partially filled 9×9 Sudoku grid (Sudoku puzzle with some cells pre-filled).

Output:
- For CSP: A completely filled 9×9 grid satisfying all Sudoku rules.
- For SA: The best grid found, which may be a near-valid solution if the algorithm does not fully converge.

Constraints (for CSP):
- Each cell contains an integer from 1–9.
- Each number appears exactly once per row, column, and 3×3 box.

Objective (for SA):
- Minimize the number of constraint violations (duplicate values in rows, columns, or boxes).
- Global optimum (score = 0) corresponds to a valid Sudoku solution.

Tasks:

- Generate Sudoku puzzles with varying difficulty levels (easy, medium, hard).
- Implement and run both algorithms on each puzzle.
    - For CSP: implement heuristics like MRV and LCV and see how they affect the results.
    - For SA: Experiment with various cooling rates and initial temperatures to observe their impact on runtime and solution quality.
- Record:
    - Solution validity (True/False): Is the solution valid?
    - Number of violations: How many constraint violations exist?
    - Runtime: Time taken to solve the puzzle
    - Iterations until convergence
- Plot and compare results
- Repeat previous tasks on *larger Sudoku variants* (e.g., 16×16) to observe scalability differences.

Submission:

- A report of up to 8 pages that describes your formulation of the problem, any heuristics that you used, how you handled constraints violations, effects of parameters tuning (at least one parameter per algorithm), experimental results and graphs, comparative discussion, and conclusions. Do not include code snippets in the report

- Source code: a well-documented implementations of both algorithms with clear input/output format

- Demo: you will be asked to present a demo of your project. During the demo, you should expect questions about your code and you might be asked to apply some minor modification.

Honor Policy: All are required to adhere to the university honor policy and violations will be dealt with according to university regulations.